Q19. DNS exchanges and TCP handshakes

Original question: Analyze a supplied packet capture or capture traffic during controlled lab communication. Identify DNS queries and responses or TCP handshake packets wherever available. Explain the observed network communication.

Aim

Identify DNS queries/responses and/or TCP handshake packets in a supplied capture or controlled lab capture, explaining their relationships.

Requirements

Wireshark/TShark and a capture; alternatively permission to capture on the selected lab interface and generate controlled traffic. This focuses on paired exchanges and sequence logic, unlike Q13's protocol inventory. References: [R12]-[R14].

Procedure

Step 1 - Open supplied evidence, or capture controlled traffic

For supplied evidence use File > Open. For a new permitted capture: open Capture > Options, choose the interface carrying the lab route, start, generate the approved traffic below, then stop and save as q19_lab.pcapng. Capture privileges must already be configured; use the lab's normal capture group/dumpcap setup rather than running the entire GUI as root.

```
# Optional traffic generation only for an approved lab environment:
DOMAIN='<AUTHORIZED_DOMAIN>'
LAB_URL='http://<LAB_TARGET_IP>:<LAB_HTTP_PORT>/'
dig "$DOMAIN" A
curl --connect-timeout 5 --max-time 10 "$LAB_URL" -o /dev/null
```

Set LAB_URL to a known lab web server. Start capture before these commands. dig generates an ordinary DNS lookup; curl can generate a fresh TCP connection. Proxies can change the actual endpoint; record their presence. A missing DNS response or failed TCP connection is an outcome to analyze, not a reason to invent packets. On loopback select lo; VM traffic may use a virtual interface. Limit capture scope to authorized traffic and save securely.

Step 2 - Find a DNS query/response pair

```
dns.flags.response == 0
dns.flags.response == 1
dns && dns.id == 0x1234
```

The ID value above is illustrative: substitute the transaction ID actually observed. Select a query and inspect transaction ID, qname, qtype, source/destination and transport ports. Find its response with matching ID, question and reversed endpoints/ports in the appropriate conversation/time window. IDs can repeat, so an ID match alone is insufficient. Read response code (NOERROR/NXDOMAIN/etc.), answers and TTL. NOERROR with no relevant answer is not the same as a positive address answer. DNS may use TCP; encrypted DNS may not be dissected as ordinary dns.

Step 3 - Find the TCP three-way handshake

```
# Display filters; enter the expression, not these comments:
tcp.flags.syn == 1 && tcp.flags.ack == 0
tcp.flags.syn == 1 && tcp.flags.ack == 1
tcp.flags.ack == 1 && tcp.flags.syn == 0 && tcp.len == 0
```

Select the first SYN and note its tcp.stream number. Apply `tcp.stream == 0` after replacing 0 with that observed stream; inspect the packets in time order. Match the same endpoint/port tuple in both directions and their sequence/acknowledgement values. The third filter finds many ordinary pure ACKs and does not

uniquely identify the final handshake ACK. A handshake ACK can carry data; if none appears, inspect all packets in the chosen stream rather than insisting tcp.len must be zero.

Step 4 - Save a terminal trace and preserve input

```
mkdir -p "$HOME/EHF_Exam/Q19"
cd "$HOME/EHF_Exam/Q19"
PCAP='<PCAP_FILE>'
sha256sum "$PCAP" | tee capture_before.sha256
STREAM='<OBSERVED_TCP_STREAM_NUMBER>'
tshark -n -r "$PCAP" -Y "tcp.stream == $STREAM" \
  -T fields -E header=y \
  -e frame.number -e ip.src -e ip.dst -e ipv6.src -e ipv6.dst \
  -e tcp.srcport -e tcp.dstport -e tcp.flags \
  -e tcp.seq -e tcp.ack -e tcp.len > handshake.tsv
cat handshake.tsv
sha256sum -c capture_before.sha256
```

If no TCP stream is present, omit this export and document the DNS exchange instead, as allowed by the question. TShark sequence numbers commonly use relative numbering; record the display setting.

Expected output - illustrative exchange

```
DNS: client:53000 -> resolver:53 | ID 0x1234 | A lab.example.org
DNS: resolver:53 -> client:53000 | ID 0x1234 | answer or error
TCP: client -> server | SYN     | seq=0
TCP: server -> client | SYN,ACK | seq=0 ack=1
TCP: client -> server | ACK     | seq=1 ack=1
```

Relative numbering is illustrated; actual initial sequence numbers differ. SYN consumes one sequence number. The server acknowledges the client's SYN, then the client acknowledges the server's SYN. Retransmissions can produce repeated SYNs and the capture may start midstream.

Interpretation, observation and result

Record DNS query/response frame numbers, transaction/question, endpoint tuple, reply code and answer. For a handshake record stream, three frame numbers, endpoints/ports, flags, seq/ack and timing. Missing handshake packets can reflect capture placement/loss or an incomplete connection; a later ACK alone is not proof that all three packets were captured. The result explains the exchange actually supported by the trace.

Common errors

- Captured on wrong interface: use routing information and traffic activity to choose the correct one.
- Many ACKs: narrow to the SYN's tcp.stream and compare sequence values.
- No DNS: cached application lookups, encrypted DNS or capture location may explain absence; controlled dig should be observed on its actual resolver path if captured.

Viva questions

- What are the three handshake stages? SYN, SYN-ACK, ACK.
- Why is the acknowledgement one larger after SYN? SYN consumes a sequence number.
- Is every ACK the third handshake packet? No; TCP uses ACK throughout a connection.
- Is matching DNS ID sufficient to pair packets? No; also check question, endpoints, ports and timing.